ATS ULP V1 – Homing Model Dokumentation

Florian

22. Mai 2026

1 Projektstruktur

Das Projekt ist als kleines Python-Paket organisiert. Die Kernlogik liegt in den Modulen unter `homing_core/`. Der alte Einstiegspunkt `homing_v4.py` bleibt als Wrapper erhalten.

2 Code-Referenzen

2.1 Datenmodelle

```python
"""Data models used by the homing implementation."""

from dataclasses import dataclass
from typing import List, NamedTuple, Optional, Tuple

import numpy as np


class FeaturePair(NamedTuple):
    """Matched snapshot/current feature pair and its differences."""

    snapshot_idx: int
    retina_idx: int
    angle_diff: float
    arc_len_diff: float


@dataclass
class RetinaFeature:
    center_angle: float
    arc_length: float
    is_landmark: bool


class Retina:
    def __init__(self, position: Tuple[float, float], features: List[RetinaFeature] =
    ↪  None):
        self.position = np.array(position, dtype=float)
        self.features: List[RetinaFeature] = features if features is not None else []

    def add_feature(self, feature: RetinaFeature):
        self.features.append(feature)


class World:
    def __init__(self):
        self.home = (0.0, 0.0)
```

```
        self.start = (-3.0, -1.0)
        self.landmarks = [
            {"pos": (3.5, 2.0), "radius": 0.5},
            {"pos": (3.5, -2.0), "radius": 0.5},
            {"pos": (0.0, -4.0), "radius": 0.5},
        ]
        self.home_retina: Optional[Retina] = None
        self.retinas: List[Retina] = []
```

2.2 Geometrie

```
"""Geometric helpers for projecting landmarks into retinal features."""

import math
from typing import Tuple

from .models import Retina, RetinaFeature, World


def angle_difference(a: float, b: float) -> float:
    return (a - b + math.pi) % (2.0 * math.pi) - math.pi


def project_landmark(pos: Tuple[float, float], lm: dict) -> RetinaFeature:
    dx = lm["pos"][0] - pos[0]
    dy = lm["pos"][1] - pos[1]
    dist = math.hypot(dx, dy)
    theta = math.atan2(dy, dx)
    ratio = min(1.0, max(-1.0, lm["radius"] / dist if dist > 1e-9 else 1.0))
    return RetinaFeature(theta, 2.0 * math.asin(ratio), True)


def build_retina(world: World, pos: Tuple[float, float]) -> Retina:
    retina = Retina(pos)
    landmarks = [project_landmark(pos, lm) for lm in world.landmarks]
    landmarks.sort(key=lambda feature: feature.center_angle)

    count = len(landmarks)
    for index in range(count):
        current_feature, next_feature = landmarks[index], landmarks[(index + 1) %
        ↪  count]
        right_edge = (current_feature.center_angle + current_feature.arc_length / 2.0)
        ↪  % (2.0 * math.pi)
        left_edge = (next_feature.center_angle - next_feature.arc_length / 2.0) % (2.0
        ↪  * math.pi)
        gap_arc = (left_edge - right_edge) % (2.0 * math.pi)
        gap = RetinaFeature((right_edge + gap_arc / 2.0) % (2.0 * math.pi), max(0.0,
        ↪  gap_arc), False)
        retina.add_feature(current_feature)
        retina.add_feature(gap)

    return retina
```

2.3 Pairing und Vektoren

```
"""Feature matching and homing vector computation."""

import math
import warnings
```

```python
from typing import List, Tuple

import numpy as np

from .geometry import angle_difference
from .models import FeaturePair, Retina, RetinaFeature


def match_feature(snapshot_idx: int, snap: RetinaFeature, retinas: List[RetinaFeature]) -> FeaturePair:
    candidates = [(index, feature) for index, feature in enumerate(retinas) if feature.is_landmark == snap.is_landmark]
    if not candidates:
        return FeaturePair(snapshot_idx, 0, 0.0, 0.0)

    best_index, best_diff = candidates[0][0], float("inf")
    for index, feature in candidates:
        diff = abs(angle_difference(snap.center_angle, feature.center_angle))
        if diff < best_diff:
            best_diff, best_index = diff, index

    matched = retinas[best_index]
    angle_diff = angle_difference(matched.center_angle, snap.center_angle)
    arc_len_diff = matched.arc_length - snap.arc_length
    return FeaturePair(snapshot_idx, best_index, angle_diff, arc_len_diff)


def compute_vectors(snapshot: Retina, current: Retina):
    """Compute turn and approach vector sums together with the matched pairs."""
    Vt, Vp = np.zeros(2), np.zeros(2)
    pairs = []

    snapshot_landmarks = sum(1 for feature in snapshot.features if feature.is_landmark)
    current_landmarks = sum(1 for feature in current.features if feature.is_landmark)
    snapshot_gaps = len(snapshot.features) - snapshot_landmarks
    current_gaps = len(current.features) - current_landmarks
    if snapshot_landmarks != current_landmarks or snapshot_gaps != current_gaps:
        warnings.warn(
            "Feature count mismatch between snapshot and current retina: "
            f"snapshot landmarks/gaps = {snapshot_landmarks}/{snapshot_gaps}, "
            f"current landmarks/gaps = {current_landmarks}/{current_gaps}. "
            "Pairing continues, but results may be unreliable.",
            RuntimeWarning,
            stacklevel=2,
        )

    for index, feature in enumerate(snapshot.features):
        pair = match_feature(index, feature, current.features)
        pairs.append(pair)

        theta = current.features[pair.retina_idx].center_angle
        tangential = np.array([-math.sin(theta), math.cos(theta)])
        radial = np.array([math.cos(theta), math.sin(theta)])

        Vt += pair.angle_diff * tangential
        Vp += (-pair.arc_len_diff) * radial

    return Vt, Vp, pairs
```

```python


def homing_vector(snapshot: Retina, current: Retina) -> np.ndarray:
    Vt, Vp, _ = compute_vectors(snapshot, current)
    vector = Vt + 3.0 * Vp
    norm = np.linalg.norm(vector)
    return vector / norm if norm > 1e-10 else np.zeros(2)
```

2.4 Visualisierung

```python
"""Matplotlib visualization for the homing model."""

import math

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Arc, Wedge

from .pairing import compute_vectors, homing_vector
from .geometry import build_retina
from .models import Retina, World


class HomingDiashow:
    """Interactive step-by-step visualization of the homing process."""

    def __init__(self, world: World, snapshot: Retina, start, step_size=0.4,
    ↪  tolerance=0.15):
        self.world = world
        self.snapshot = snapshot
        self.path = [np.array(start, dtype=float)]
        self.step_size = step_size
        self.tolerance = tolerance

        self.r_snap = 0.50
        self.r_cur = 1.00

        self.state = 0
        self.labels = ["Retinas", "Pairing", "Komponenten", "Vektor", "Bewegung"]

        self.fig, self.ax = plt.subplots(figsize=(10, 10))
        self.fig.canvas.mpl_connect("key_press_event", self.on_key)
        self._draw()
        plt.show()

    def _draw_ring(self, center, retina: Retina, radius: float, hollow=False):
        for feature in retina.features:
            angle_deg = np.degrees(feature.center_angle)
            half_width = max(np.degrees(feature.arc_length / 2.0), 1.0)
            color = "red" if feature.is_landmark else "green"

            if hollow:
                arc = Arc(center, 2 * radius, 2 * radius, angle=0, theta1=angle_deg -
                ↪  half_width, theta2=angle_deg + half_width, color=color,
                ↪  linewidth=10, zorder=10)
                self.ax.add_patch(arc)
            else:
```

```
                wedge = Wedge(center, radius, angle_deg - half_width, angle_deg +
                ↪  half_width, facecolor=color, edgecolor="black", linewidth=1,
                ↪  alpha=0.8, zorder=10)
                self.ax.add_patch(wedge)

    def _draw(self):
        self.ax.clear()
        position = self.path[-1]
        current = build_retina(self.world, tuple(position))

        for landmark in self.world.landmarks:
            self.ax.add_patch(plt.Circle(landmark["pos"], landmark["radius"],
            ↪  color="skyblue", ec="navy", zorder=5))
        self.ax.plot(0, 0, "kX", markersize=12, zorder=6, label="Home")
        if len(self.path) > 1:
            path = np.array(self.path)
            self.ax.plot(path[:, 0], path[:, 1], "r.-", linewidth=2, zorder=4,
            ↪  label="Pfad")
        else:
            self.ax.plot(position[0], position[1], "mo", markersize=8, zorder=6,
            ↪  label="Start")

        if self.state in (0, 1, 2, 3):
            self._draw_ring(position, self.snapshot, self.r_snap, hollow=False)
            self._draw_ring(position, current, self.r_cur, hollow=True)

        if self.state == 1:
            _, _, pairs = compute_vectors(self.snapshot, current)
            for pair in pairs:
                snapshot_feature = self.snapshot.features[pair.snapshot_idx]
                retina_feature = current.features[pair.retina_idx]
                color = "red" if snapshot_feature.is_landmark else "green"
                x1 = position[0] + self.r_snap *
                ↪  math.cos(snapshot_feature.center_angle)
                y1 = position[1] + self.r_snap *
                ↪  math.sin(snapshot_feature.center_angle)
                x2 = position[0] + self.r_cur * math.cos(retina_feature.center_angle)
                y2 = position[1] + self.r_cur * math.sin(retina_feature.center_angle)
                self.ax.plot([x1, x2], [y1, y2], color=color, linewidth=2.5,
                ↪  alpha=0.8, zorder=11)
                self.ax.plot([x1, x2], [y1, y2], "o", color=color, markersize=4,
                ↪  zorder=12)

        if self.state == 2:
            turn_total, approach_total, pairs = compute_vectors(self.snapshot,
            ↪  current)

            for pair in pairs:
                retina_feature = current.features[pair.retina_idx]
                theta = retina_feature.center_angle
                start_x = position[0] + self.r_cur * math.cos(theta)
                start_y = position[1] + self.r_cur * math.sin(theta)

                tangential = np.array([-math.sin(theta), math.cos(theta)])
                radial = np.array([math.cos(theta), math.sin(theta)])

                turn_vector = pair.angle_diff * tangential
                approach_vector = (-pair.arc_len_diff) * radial

```

```
                self.ax.quiver(start_x, start_y, turn_vector[0], turn_vector[1],
                ↪  scale=3, color="purple", width=0.006, headwidth=4, headlength=5,
                ↪  zorder=15, alpha=0.9)
                self.ax.quiver(start_x, start_y, approach_vector[0],
                ↪  approach_vector[1], scale=3, color="orange", width=0.006,
                ↪  headwidth=4, headlength=5, zorder=15, alpha=0.9)

            self.ax.quiver(position[0], position[1], turn_total[0], turn_total[1],
            ↪  scale=3, color="deepskyblue", width=0.01, headwidth=5, headlength=6,
            ↪  zorder=16, alpha=0.55, label="Vt (Turn, Summe)")
            self.ax.quiver(position[0], position[1], approach_total[0],
            ↪  approach_total[1], scale=3, color="saddlebrown", width=0.01,
            ↪  headwidth=5, headlength=6, zorder=16, alpha=0.55, label="Vp (Approach,
            ↪  Summe)")

        if self.state == 3:
            vector = homing_vector(self.snapshot, current)
            self.ax.quiver(position[0], position[1], vector[0], vector[1], scale=5,
            ↪  color="black", width=0.008, zorder=15)

        self.ax.set_xlim(-7.5, 7.5)
        self.ax.set_ylim(-7.5, 7.5)
        self.ax.set_aspect("equal")
        self.ax.grid(True, linestyle="--", alpha=0.3)
        self.ax.set_title(f"Schritt {len(self.path)-1} | {self.labels[self.state]} |
        ↪  Pos: ({position[0]:.2f}, {position[1]:.2f}) - LEERTASTE")
        self.ax.legend(loc="upper left")
        self.fig.canvas.draw_idle()

    def on_key(self, event):
        if event.key != " ":
            return

        if self.state == 4:
            position = self.path[-1]
            home = np.array(self.world.home)
            distance = np.linalg.norm(position - home)
            if distance < self.tolerance:
                self.ax.set_title("ZIEL ERREICHT", fontsize=16, color="green")
                self.fig.canvas.draw_idle()
                return

            current = build_retina(self.world, tuple(position))
            vector = homing_vector(self.snapshot, current)
            step = min(self.step_size, distance * 0.8)
            self.path.append(position + vector * step)
            self.state = 0
        else:
            self.state += 1

        self._draw()
```

2.5 Demo-Start

```
"""Executable demo entrypoint for the homing package."""

import numpy as np
import matplotlib.pyplot as plt

```

```
from .geometry import build_retina
from .models import World
from .pairing import homing_vector
from .visualization import HomingDiashow


def _precompute_path(world: World, snapshot, step_size: float, tolerance: float,
    ↪  max_steps: int = 50):
    path = [np.array(world.start, dtype=float)]
    for _ in range(max_steps):
        position = path[-1]
        distance = np.linalg.norm(position - np.array(world.home))
        if distance < tolerance:
            break
        current = build_retina(world, tuple(position))
        vector = homing_vector(snapshot, current)
        path.append(position + vector * min(step_size, distance * 0.8))
    return path


def _build_vector_field(world: World, snapshot):
    x_values = np.arange(-7, 8)
    y_values = np.arange(-7, 8)
    X, Y = np.meshgrid(x_values, y_values)
    U, V = np.zeros_like(X, dtype=float), np.zeros_like(Y, dtype=float)

    for row in range(X.shape[0]):
        for col in range(X.shape[1]):
            px, py = float(X[row, col]), float(Y[row, col])
            if abs(px) < 0.01 and abs(py) < 0.01:
                continue
            retina = build_retina(world, (px, py))
            vector = homing_vector(snapshot, retina)
            U[row, col], V[row, col] = vector[0], vector[1]

    return X, Y, U, V


def run_demo():
    world = World()
    snapshot = build_retina(world, world.home)
    world.home_retina = snapshot

    step_size, tolerance = 0.4, 0.15
    path = _precompute_path(world, snapshot, step_size, tolerance)

    plt.ion()
    fig, ax = plt.subplots(figsize=(9, 9))
    X, Y, U, V = _build_vector_field(world, snapshot)
    ax.quiver(X, Y, U, V, scale=25, width=0.003, color="black")

    for landmark in world.landmarks:
        ax.add_patch(plt.Circle(landmark["pos"], landmark["radius"], color="skyblue",
        ↪  ec="navy", zorder=5))

    ax.plot(0, 0, "kX", markersize=12, zorder=6, label="Home")
    ax.plot(world.start[0], world.start[1], "mo", markersize=8, zorder=6,
    ↪  label="Start")

```

```
62      path_array = np.array(path)
63      ax.plot(path_array[:, 0], path_array[:, 1], "r-", linewidth=2.5, zorder=4,
        ↪  label="Suchpfad")
64      ax.plot(path_array[:, 0], path_array[:, 1], "r.", markersize=5, zorder=4)
65
66      ax.set_xlim(-7.5, 7.5)
67      ax.set_ylim(-7.5, 7.5)
68      ax.set_aspect("equal")
69      ax.grid(True, linestyle="--", alpha=0.3)
70      ax.set_title("Homing-Vektorfeld")
71      ax.legend(loc="upper left")
72      fig.canvas.draw()
73      fig.show()
74
75      plt.ioff()
76      print("=" * 50)
77      print("Zwei Fenster geoffnet:")
78      print("  [1] Vektorfeld (statisch)")
79      print("  [2] Diashow (interaktiv - hier klicken + LEERTASTE)")
80      print("=" * 50)
81
82      HomingDiashow(world, snapshot, world.start, step_size, tolerance)
83
84
85  if __name__ == "__main__":
86      run_demo()
```

3 Screenshots

Alle Screenshots für die Dokumentation werden in `docs/screenshots/` abgelegt und von dort eingebunden.

Screenshot-Platzhalter. Datei in `docs/screenshots/` ablegen und hier mit `\includegraphics` referenzieren.

Abbildung 1: Platzhalter für Visualisierungen aus dem Homing-Projekt.